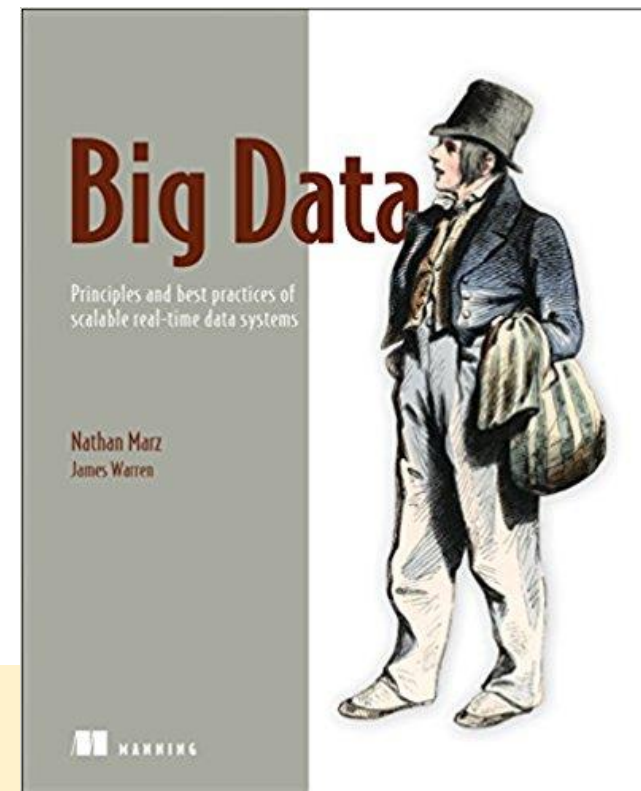




3rd Lesson – BATCH LAYER



Recap Lambda Architecture

Non dimenticate di approfondire... ad esempi, kappa architecture!

Lambda Architecture



Batch Layer

- Stores the immutable constantly growing dataset
- Computes arbitrary views from this dataset using BigData technologies (can take hours)
- Can be always recreated

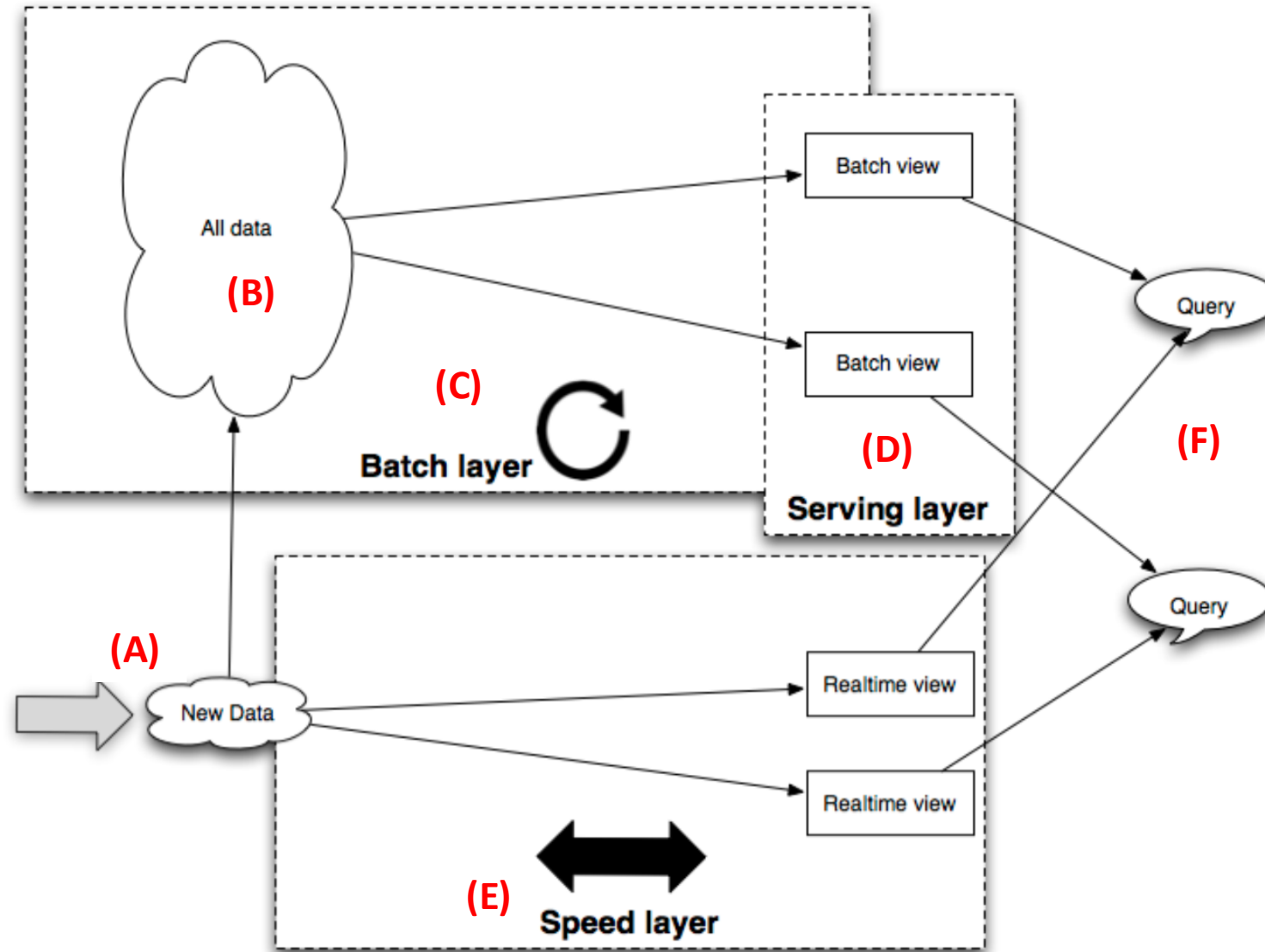
Serving Layer

- Indexing and exposing
 - the pre-computed batch views
 - the incremented real-time views
- Merges the batch and the real-time views

Speed Layer

- Computes views from the data stream in real-time
- Compensate for the high latency of the batch layer
- Incremental model and views are transient

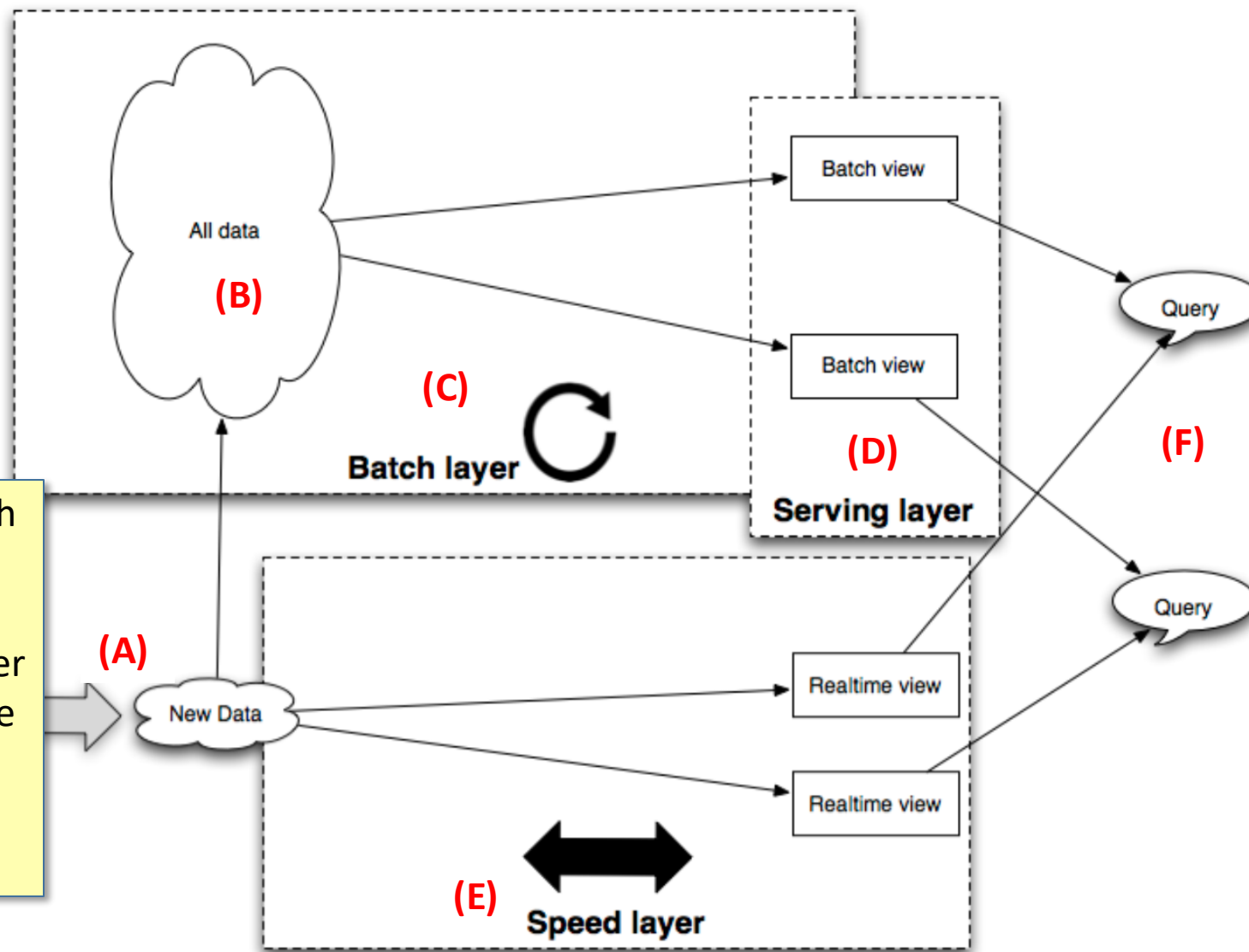
Lambda Architecture



Lambda Architecture



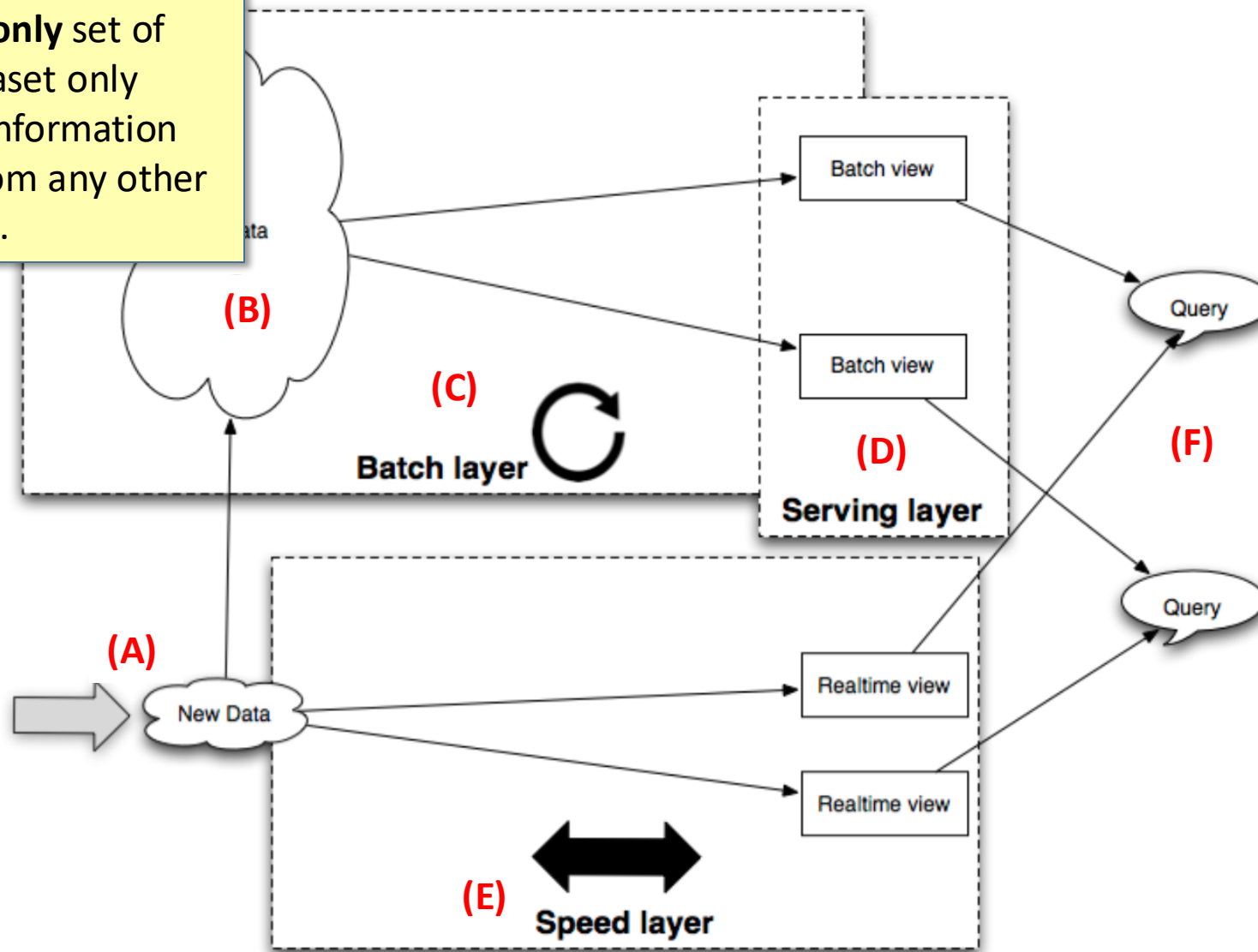
(A) All new data is sent to both the batch layer and the speed layer. In the batch layer, new data is appended to the master dataset. In the speed layer, the new data is consumed to do incremental updates of the realtime views.



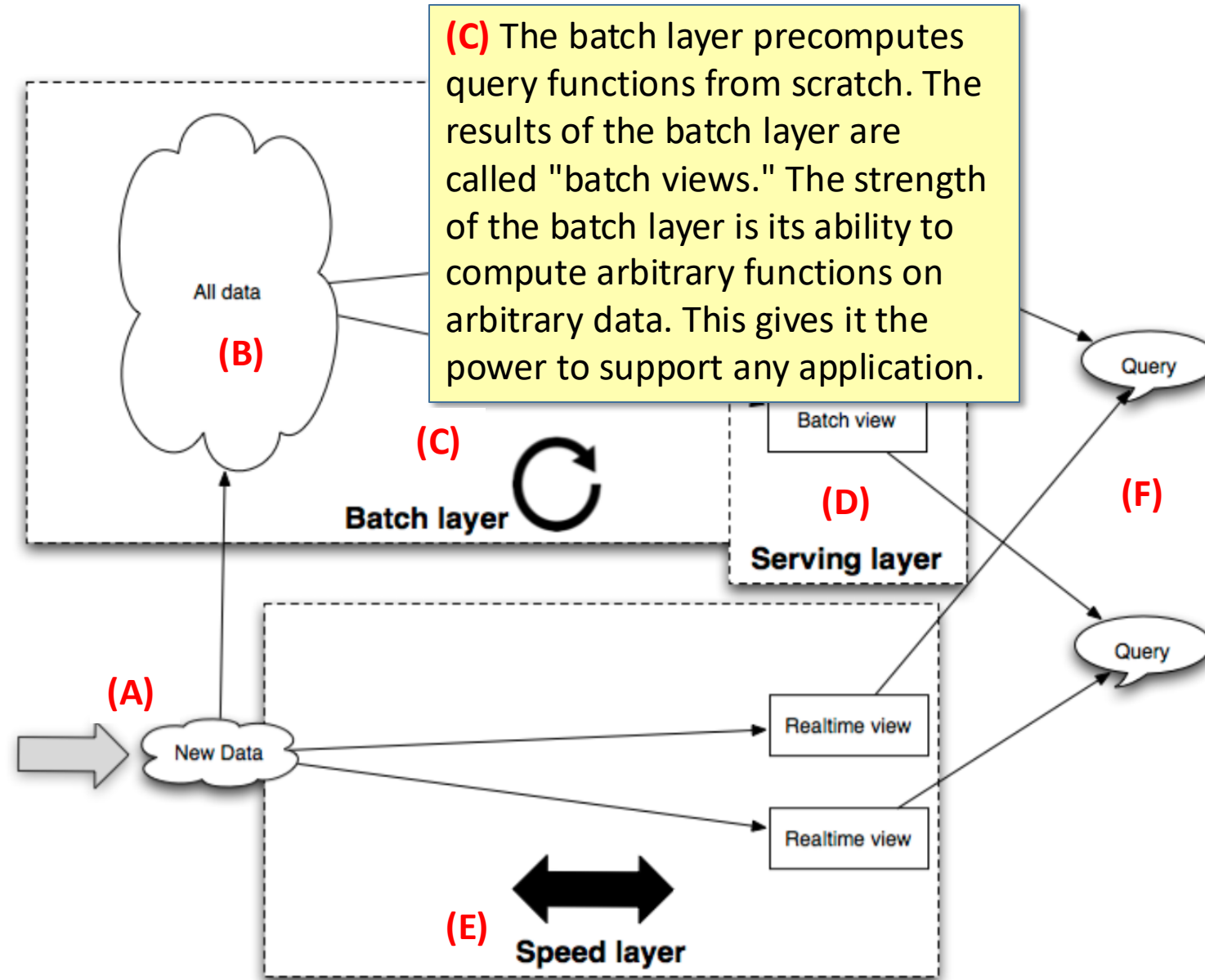
Lambda Architecture



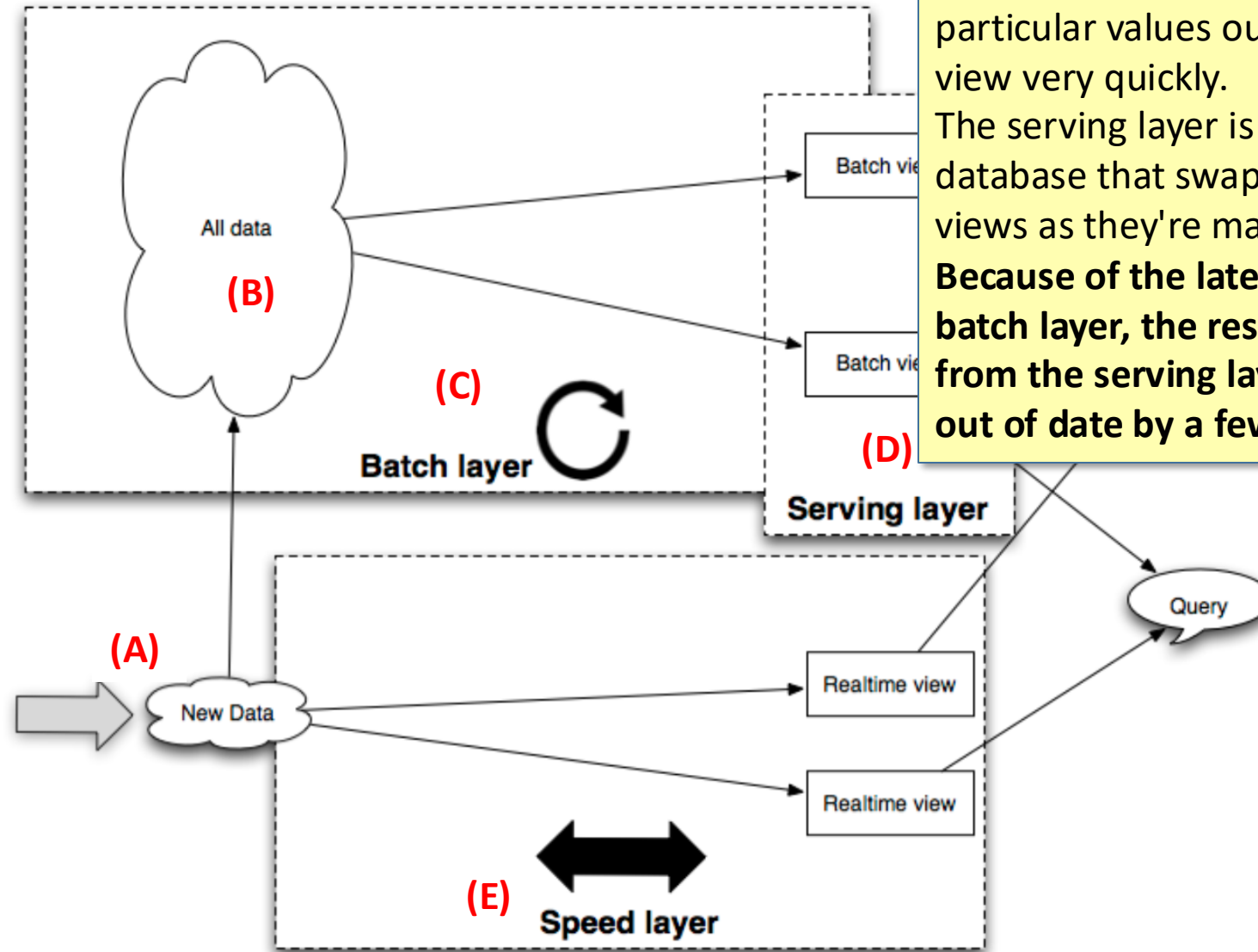
(B) The master dataset is an **immutable, append-only** set of data. The master dataset only contains the **rawest** information that is not derived from any other information you have.



Lambda Architecture

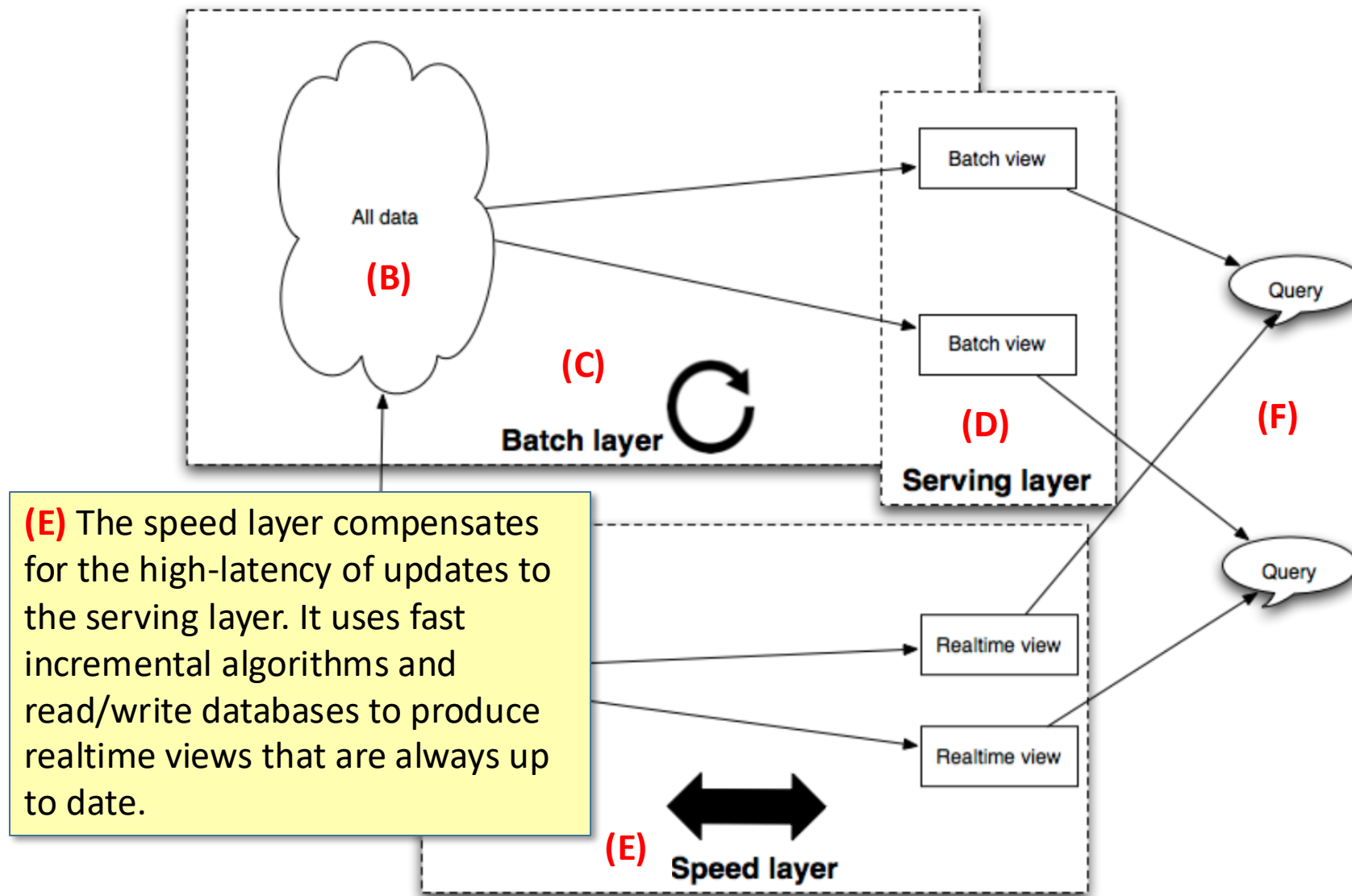


Lambda Architecture

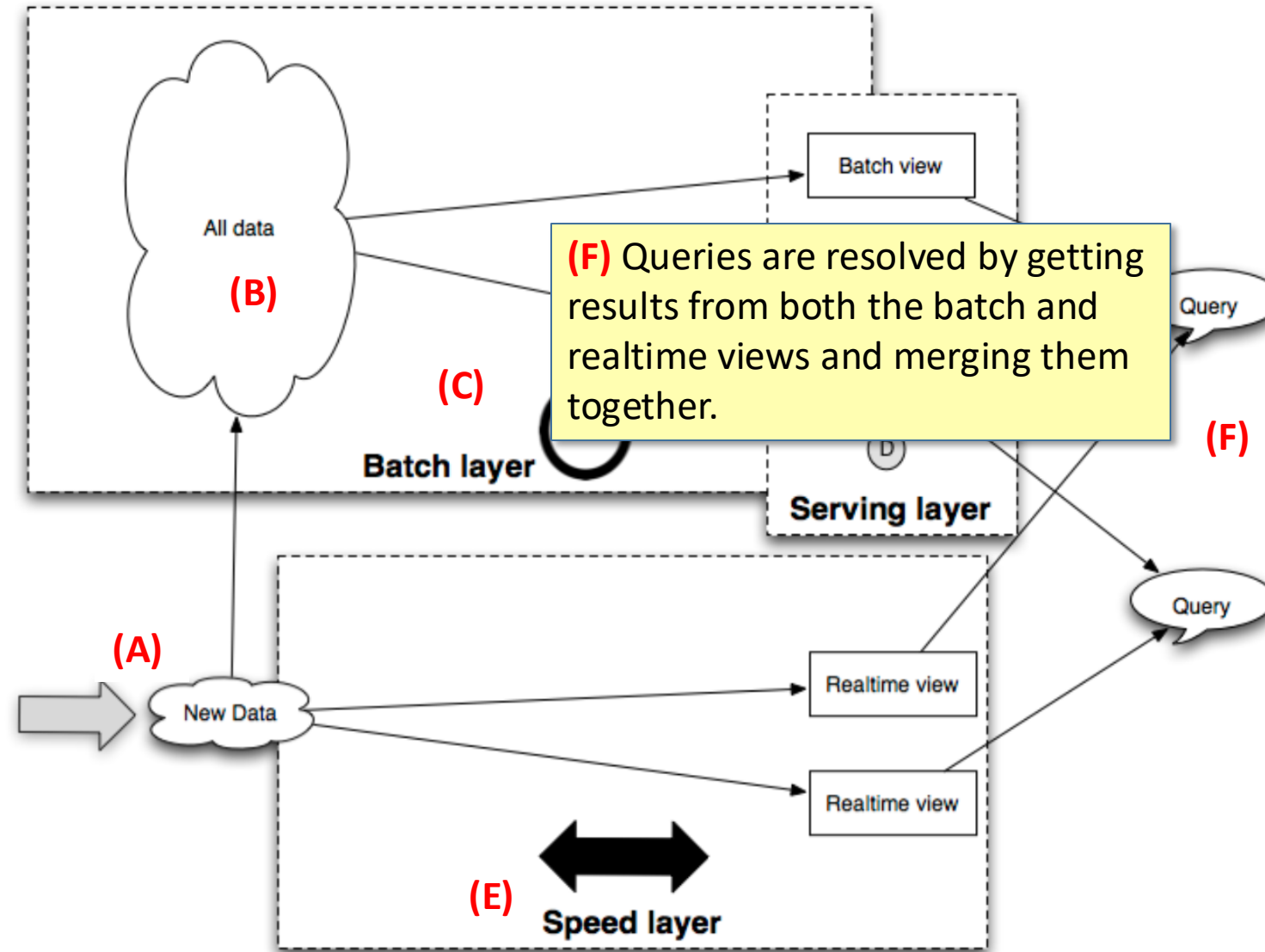


(D) The serving layer indexes the batch views produced by the batch layer and makes it possible to get particular values out of a batch view very quickly. The serving layer is a scalable database that swaps in new batch views as they're made available. **Because of the latency of the batch layer, the results available from the serving layer are always out of date by a few hours.**

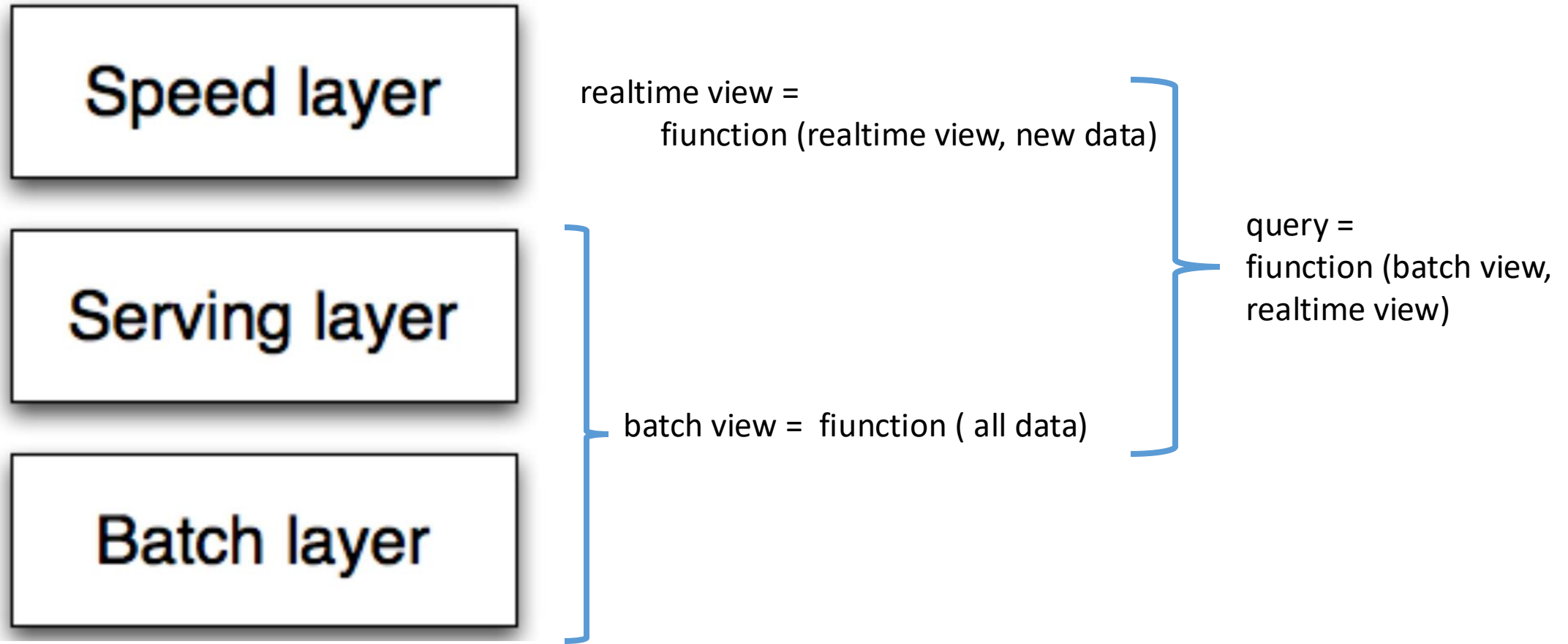
Lambda Architecture



Lambda Architecture



Lambda Architecture





Big Data File Formats

- How to choose the right data format
 - Row or Column based storage
 - Schema evolution
 - Splitability
 - Compression

FORMAT	COLUMNAR	COMPRESSION	SUPPORT
AVRO	X	GOOD	HADOOP SPARK ATHENA PRESTO
PARQUET	✓	GREAT	HADOOP SPARK ATHENA PRESTO
ORC	✓	EXCELLENT	HADOOP SPARK ATHENA PRESTO
CARBONDATA	✓	GOOD	HADOOP SPARK

- *"The analyst and data scientist may gain more insight into why different formats emerged, and some of the trade-offs required when choosing a format"*
- *The data engineer might better understand the evolution of data formats and ideal use cases for each type*
- *The business user will be able to understand why their analysts and engineers may prefer certain formats—and what "Avro," "Parquet," and "ORC" mean!"*

Quoted by White paper "An introduction to big data formats", NEXLA

Please, don't forget to read supplement material provided by the tacher

- Faster read times
- Faster write times
- Splittable files (so you don't need to read the whole file, just a part of it)
- Schema evolution support (allowing you to change the fields in a dataset)
- Advanced compression support (compress the files with a compression codec without sacrificing these features)

Row vs Column



- **Row-based** is most suited when your queries require access to all or most of the columns of each row of data
- **Column-based** storage is most useful when performing analytics queries that require only a subset of columns examined over very large data sets;

ROW-BASED REPRESENTATION



COLUMN-BASED REPRESENTATION



Row vs Column



- **Row-based**

```
Emma, Prod1, 100.00, 2018-04-02; Liam, Prod2, 79.99, 2018-04-02; Noah, Prod3, 19.99, 2018-04-01; Olivia, Prod2, 79.99, 2018-04-03
```

- **Column-based**

```
Emma, Liam, Noah, Olivia; Prod1, Prod2, Prod3; Prod2; 100.00, 79.99, 19.99, 79.99; 2018-04-02, 2018-04-02, 2018-04-01, 2018-04-03
```

Customer Name	Product ID	Sale Amount	Transaction Date
Emma	Prod 1	100.00	2018-04-02
Liam	Prod 2	79.99	2018-04-02
Noah	Prod 3	19.99	2018-04-01
Olivia	Prod 2	79.99	2018-04-03

- How easy is it to update a schema (such as adding a field, removing or re-naming a field)?
- How will different versions of the schema “talk” to each other?
- Is it human-readable? Does it need to be?
- How fast can the schema be processed?
- How does it impact the size of data?

- Row-based formats, such as Avro, can be split along row boundaries, as long as the processing can proceed with one record at a time. If groups of records related by some particular column value are required for processing, out-of-the box partitioning may be more challenging for row-based data stored in random order.
- A column-based format will be more amenable to splitting into separate jobs if the query calculation is concerned with a single column at a time. The columnar formats take a batch of rows and store that batch in columnar format. These batches then become split boundaries.

Compression



- While compression may save on storage costs, it is important to also consider compute costs and resources. Chances are, at some point you will want to decompress that data for use in another application. Decompression is not free—it incurs compute costs. If and how you compress the data will be a function of how you want to optimize the compute costs vs. storage costs for your given use case.

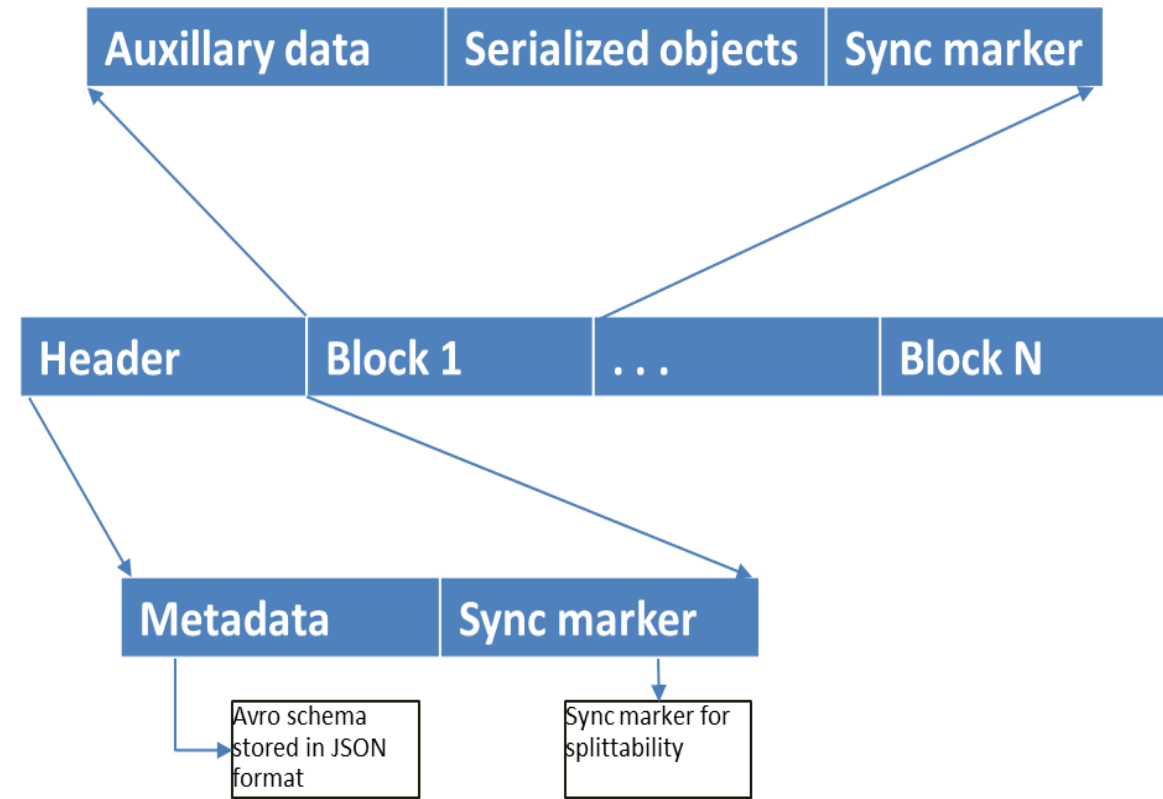
- Plain text storage (eg, **CSV**, TSV files)
- Row-based
 - ❖ **Avro**
- Column-based
 - ❖ **Parquet**
 - ❖ **ORC**
- Thrift
- Sequence Files

- Language neutral data serialization system
 - Write a file in python and read it in C
- AVRO data is described using language independent schema
- AVRO schemas are usually written in JSON and data are encoded in binary format
- Supports schema evolution
 - producers and consumers at different versions of schema
- Supports compression and are splitability
- Avro stores the schema in header of file so data is *self-describing*

AVRO – File structure and example



```
{  
  "type" : "record",  
  "name" : "tweets",  
  "fields" : [ {  
    "name" : "username",  
    "type" : "string",  
  }, {  
    "name" : "tweet",  
    "type" : "string",  
  }, {  
    "name" : "timestamp",  
    "type" : "long",  
  } ],  
  "doc:" : "schema for storing tweets"  
}
```



AVRO – From json example



```
student.avsc
{
  "type" : "record",
  "name" : "student_marks",
  "namespace" : "com.rishav.avro",
  "fields" : [ {
    "name" : "student_id",
    "type" : "int"
  }, {
    "name" : "subject_id",
    "type" : "int"
  }, {
    "name" : "marks",
    "type" : "int"
  } ]
}
```

```
student.json
{"student_id":1,"subject_id":63,"marks":19}
{"student_id":2,"subject_id":64,"marks":74}
{"student_id":3,"subject_id":10,"marks":94}
{"student_id":4,"subject_id":79,"marks":27}
{"student_id":1,"subject_id":52,"marks":95}
{"student_id":2,"subject_id":34,"marks":16}
{"student_id":3,"subject_id":81,"marks":17}
{"student_id":4,"subject_id":60,"marks":52}
{"student_id":1,"subject_id":11,"marks":66}
{"student_id":2,"subject_id":84,"marks":39}
{"student_id":3,"subject_id":24,"marks":39}
{"student_id":4,"subject_id":16,"marks":0}
{"student_id":1,"subject_id":65,"marks":75}
{"student_id":2,"subject_id":5,"marks":52}
{"student_id":3,"subject_id":86,"marks":50}
{"student_id":4,"subject_id":55,"marks":42}
{"student_id":1,"subject_id":30,"marks":21}
```

student.json --schema-file student.avsc > student.avro

AVRO- Schema Evolution Example



```
{'namespace': "drwho.avro",  
  "type": "record",  
  "name": "drwho",  
  "fields": [  
    {'name': "doctor_who_season", "type": "string"}, {'name': "doctor_actor", "type": "string"},  
    {'name': "episode_no", "type": "string"}, {'name': "episode_title", "type": "string"},  
    {'name': "date_from", "type": "string"}, {'name': "date_to", "type": "string"},  
    {'name': "estimated", "type": "string"}, {'name': "planet", "type": "string"},  
    {'name': "sub_location", "type": "string"}, {'name': "main_location", "type": "string"}  
  ]  
}
```

Avro - Schema Evolution Example



```
{'namespace': 'drwho.avro',  
  'type': 'record',  
  'name': 'drwho',  
  'fields': [  
    {'name': 'drwho_season', 'type': ['null', 'string'], 'aliases': ['doctor_who_season']},  
    {'name': 'drwho_actor', 'type': ['null', 'string'], 'aliases': ['doctor_actor']},  
    {'name': 'episode_no', 'type': ['null', 'string']}, {'name': 'episode_title', 'type': ['null', 'string']},  
    {'name': 'date_from', 'type': ['null', 'string']}, {'name': 'date_to', 'type': ['null', 'string']},  
    ['name': 'estimated', 'type': 'string'], {'name': 'planet', 'type': ['null', 'string']},  
    {'name': 'sub_location', 'type': ['null', 'string']}, {'name': 'main_location', 'type': ['null', 'string']},  
    ['name': 'hd', 'type': 'string', 'default': 'no']  
  ]  
}
```

- Parquet is a *columnar format*. Columnar formats work well where only few columns are required in query/ analysis.
- Only required columns would be fetched / read, it *reduces the disk I/O*.
- Parquet is well suited for data warehouse kind of solutions where aggregations are required on certain column over a huge set of data.
- Parquet provides very good compression upto 75% when used with compression formats like snappy.
- Parquet can be read and write using Avro API and Avro Schema.
- It also provides *predicate pushdown*, thus reducing further disk I/O cost.

- Columnar format
- Splittable
- Stores data as group of rows and each group has columnar storage.
- Gives Indexing within row group.
- Not general purpose as it was designed to perform with Hive. We cannot integrate it with query engine like Impala.

Big Data Formats Comparison



	Avro	Parquet	ORC
Schema Evolution Support			
Compression			
Splitability			
Most Compatible Platforms	Kafka, Druid	Impala, Arrow Drill, Spark	Hive, Presto
Row or Column	Row	Column	Column
Read or Write	Write	Read	Read

Suggested readings



- <https://www.nexla.com/resource/introduction-big-data-formats-understanding-avro-parquet-orc/>
- *File Formats Description*
 - <https://www.datanami.com/2018/05/16/big-data-file-formats-demystified/>
 - <https://blog.matthewrathbone.com/2016/09/01/a-beginners-guide-to-hadoop-storage-formats.html>
- *File Formats Performance Comparison*
 - https://www.slideshare.net/Hadoop_Summit/big-data-storage-comparing-speed-and-features-for-avro-json-orc-and-parquet
 - <https://www.slideshare.net/StampedeCon/choosing-an-hdfs-data-storage-format-avro-vs-parquet-and-more-stampedecon-2015>
- *Avro + Python*
 - <http://www.treselle.com/blog/avro-with-python-part-2/>
- *Master Dataset Partitioning Issues*
 - <http://www.datio.com/iaas/understanding-the-data-partitioning-technique/> (Take care in this link you will see different definition of MasterDataset)
 - https://jobs.zalando.com/tech/blog/solving-many-small-files-avro/?gh_src=4n3gxh1
- *More details about Parquet*
 - https://blog.twitter.com/engineering/en_us/a/2013/dremel-made-simple-with-parquet.html

Suggested Readings



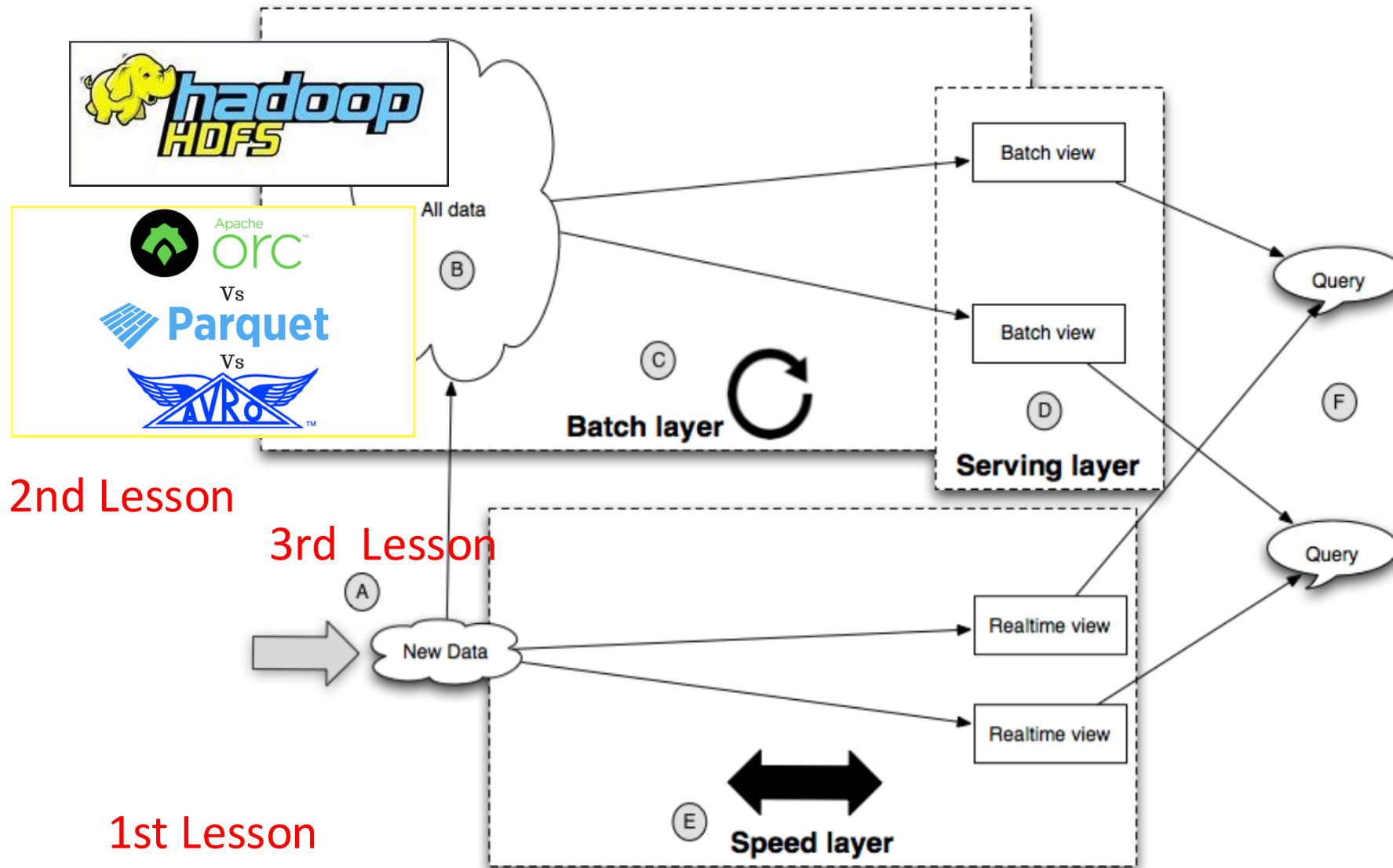
- Partitioning Issues
 - <http://www.datio.com/iaas/understanding-the-data-partitioning-technique/>
- Repartitioning
 - https://jobs.zalando.com/tech/blog/solving-many-small-files-avro/?gh_src=4n3gxh1
- *Data Partitioning and File Formats Case Study – Risk Data Analytics*
 - <https://techmagie.wordpress.com/category/big-data/avro-parquet/>



3rd Lesson - NoSQL

- NoSQL and LAMBDA Architecture

Previous Lessons



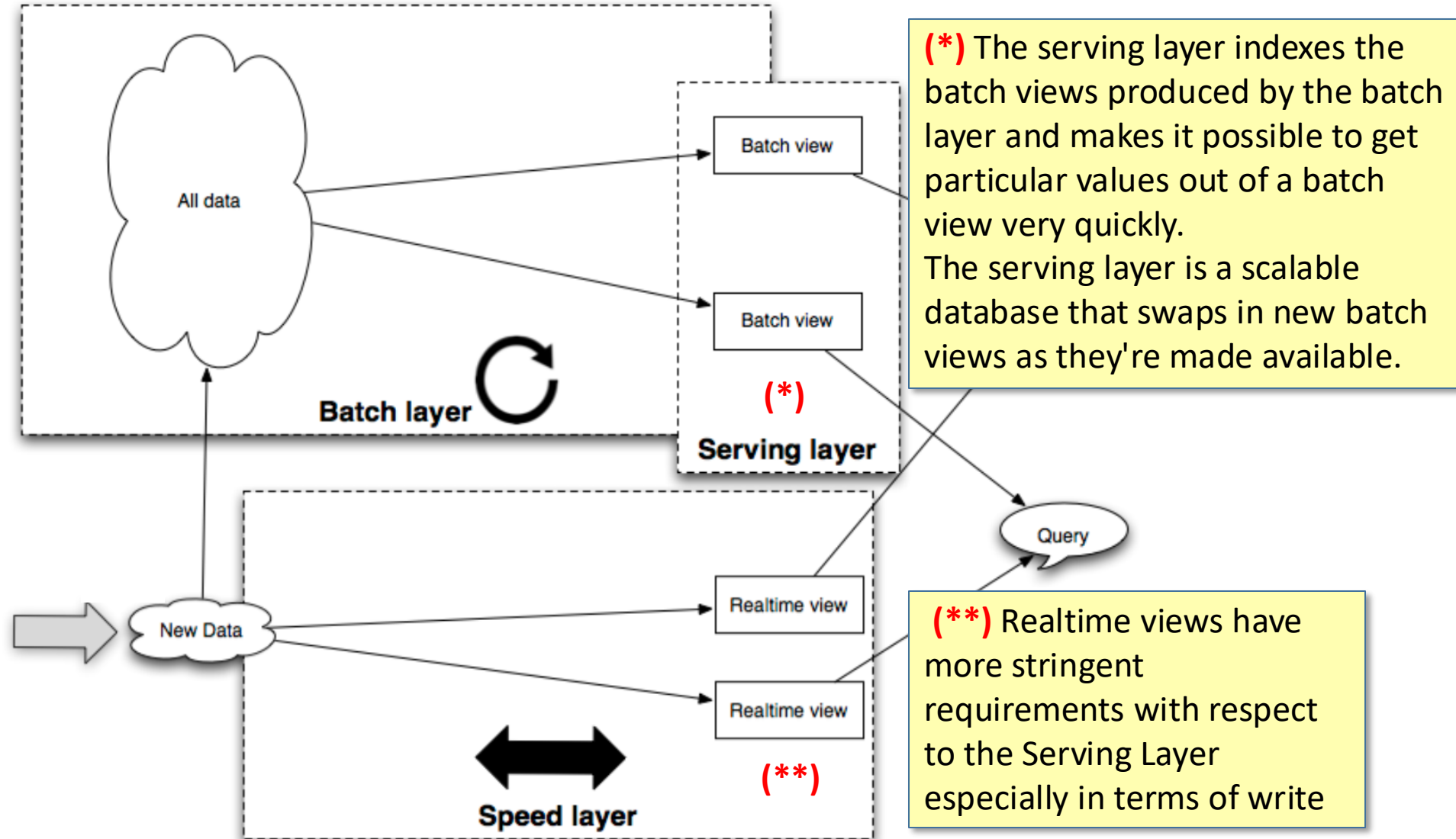
2nd Lesson

3rd Lesson

1st Lesson

Lambda Architecture

The focus of the 3rd Lesson



Lambda Architecture

Consistency vs Efficiency

- 1 Data is normalized in the master dataset for compactness and consistency...

Batch layer



Master dataset

- 3 The batch views are continually rebuilt from the master dataset, so all changes are consistent across the batch views.

- 2 ... but is redundantly stored (denormalized) in the batch views for efficient querying.

Serving layer



Batch view



Batch view

Query:

"How many 2"

Normalized vs Denormalized

Batch Layer – Master Data Storage Requirements



- **Write** — The only write operation is to add new species of data, in addition let's consider that the batch layer stores the complete dataset (terabytes, petabytes, and more)
 - Efficient appends of new data.
 - Scalable storage
- **Read** — Support for parallel processing.
- **Both** — You may choose to compress your data to help minimize your expenses. Let's consider that it's critical to guarantee the immutability property.
 - Tunable storage and processing costs.
 - Enforceable immutability

Serving Layer – Storage Requirements



- **Batch writable** — The batch views for a serving layer are produced from scratch. When a new version of a view becomes available, it must be possible to completely swap out the older version with the updated view.
- **Random reads**—A serving layer database must support random reads, with indexes providing direct access to small portions of the view. This requirement is necessary to have low latency on queries.
- **Scalable** — A serving layer database must be capable of handling views of arbitrary size. As with the distributed filesystems and batch computation, this requires it to be distributed across multiple machines.
- **Fault-tolerant**—Because a serving layer database is distributed, it must be tolerant of machine failures.

Speed Layer - Storage Requirements



- **Random writes**—To support incremental algorithms, it must also be possible to modify a realtime view with low latency.
- **Random reads**—A realtime view should support fast random reads to answer queries quickly. This means the data it contains must be indexed.
- **Scalability**—As with the serving layer views, the realtime views should scale with the amount of data they store and the read/write rates required by the application. Typically this implies that realtime views can be distributed across many machines.
- **Fault tolerance**—If a disk or a machine crashes, a realtime view should continue to function normally. Fault tolerance is accomplished by replicating data across machines so there are backups should a single machine fail.



File System

- Si è cominciato ad archiviare ed elaborare dati con i computer prima dell'avvento del database.
- Per essere utili per le applicazioni aziendali, i computer dovevano archiviare, manipolare e recuperare file di dati di grandi dimensioni.
 - A tale scopo sono stati sviluppati sistemi di elaborazione di file informatici: **file system**.
- Es: File System
 - I dati possono essere memorizzati in file excel, il file excel vengono gestiti come tutti gli altri file

Intro caso Pine Valley Furniture Company



- Pine Valley Furniture Company (PVFC) produce mobili di alta qualità e li distribuisce ai negozi al dettaglio in tutta la nazione.
- Il cliente invia gli ordini a Pine Valley Furniture con uno dei vari mezzi come telefono, posta, fax o moduli elettronici via Internet.
- Questa società ha inizialmente utilizzato sistemi basati su computer per:
 - l'elaborazione degli ordini
 - l'emissione delle fatture
 - il libro paga per il personale

- In tutte le applicazioni hanno utilizzato il tradizionale approccio di elaborazione dei file.
- Si sono concentrati sulle esigenze di elaborazione dei dati dei singoli reparti (dipartimenti).
- Ogni programma applicativo o sistema sviluppato è progettato per soddisfare le esigenze del particolare reparto o gruppo di utenti richiedente.
- Quindi non esiste una mappa, un piano o un modello generale per guidare la crescita delle applicazioni.

PVFC – file system



- Le applicazioni informatiche (basate sull'approccio di elaborazione dei file) più importanti per PVFC sono:
 - Evasione degli ordini,
 - Fatturazione (Invoicing)
 - Libro paga (payroll)

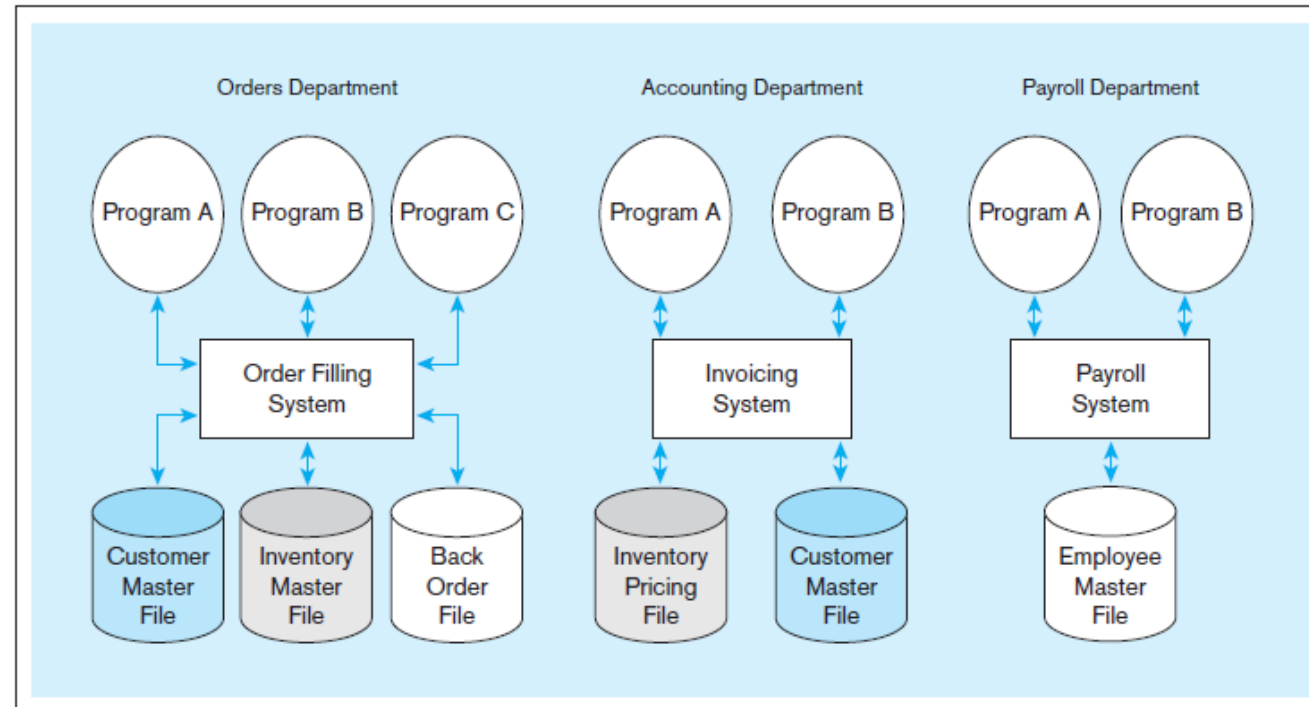


FIGURE 1-2 Old file processing systems at Pine Valley Furniture Company

La figura illustra gli svantaggi dell'utilizzo dell'approccio di elaborazione dei file in PVFC (v. slide succ.)

PVFC – file system



- Si noti che esiste una duplicazione di alcuni dei file utilizzati dalle tre applicazioni, tipico dei sistemi di elaborazione dei file.

Dati Duplicati

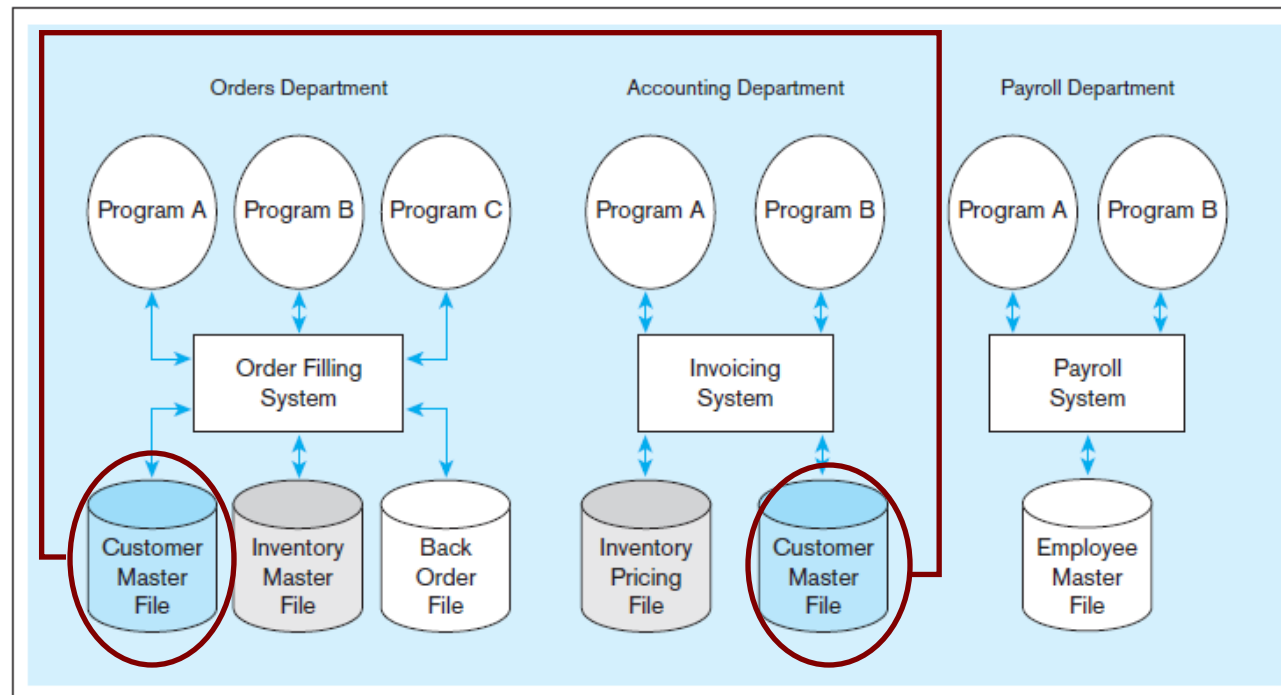


FIGURE 1-2 Old file processing systems at Pine Valley Furniture Company



Normalizzazione

Normalizzazione dei dati



- strumento per convalidare e migliorare una progettazione logica:
 - soddisfare determinati vincoli per evitare inutili duplicazioni di dati
- decomposizione delle relazioni con le anomalie per produrre relazioni più piccole e ben strutturate

Relazioni ben strutturate



- Relazione che contiene una ridondanza minima dei dati e consente agli utenti di inserire, eliminare e aggiornare righe senza causare incoerenze nei dati
- L'obiettivo è evitare anomalie

- è una tabella di dati bidimensionale con un determinato nome
- è costituita da righe (record) e colonne (attributo o campo)
- per essere qualificata come relazione:
 - Deve avere un nome univoco.
 - Ogni valore di attributo deve essere atomico (non multivalore, non composito).
 - Ogni riga deve essere univoca (non può avere due righe con esattamente gli stessi valori per tutti i relativi campi).
 - Gli attributi (colonne) della tabella devono avere nomi univoci.
 - L'ordine delle colonne deve essere irrilevante.
 - L'ordine delle righe deve essere irrilevante.

NOTA: tutte le relazioni sono nella prima forma normale.

Esempio di relazione



- La relazione EMPLOYEE1 contiene i seguenti attributi che descrivono i dipendenti: EmpID, Name, DeptName e Salary.
- Le cinque righe della tabella corrispondono a cinque dipendenti.
- Se aggiungiamo un'altra riga di dati alla figura o modifichiamo uno qualsiasi dei dati nelle righe esistenti, è sempre la stessa relazione.
- L'eliminazione di una riga non modifica la relazione.
- In effetti, potremmo eliminare tutte le righe e la relazione EMPLOYEE1 esisterebbe ancora.

Esempio di relazione



EMPLOYEE1

<u>EmpID</u>	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

Una relazione ben strutturata contiene una ridondanza minima e consente agli utenti di inserire, modificare ed eliminare le righe in una tabella senza errori o incoerenze.

EMPLOYEE1(EmpID, Name, DeptName, Salary)

- EMPLOYEE1 è una relazione ben strutturata. Ogni riga della tabella contiene i dati che descrivono un dipendente e qualsiasi modifica ai dati di un dipendente (come una modifica dello stipendio) è limitata ad una riga della tabella.
- EMPLOYEE2 non è una relazione ben strutturata (Se si esaminano i dati di esempio nella tabella, si noterà una notevole ridondanza.)

EMPLOYEE2

<u>EmpID</u>	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

Esempio



EMPLOYEE2

<u>EmpID</u>	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015

E' una relazione?

Sì: righe univoche e senza attributi multivalore

Qual è la chiave primaria?

Composita: EmpID, CourseTitle

Questa è una relazione. Ma non è una relazione ben strutturata.

Anomalie da evitare



Anomalia di **inserimento**

- aggiunta di nuove righe → creazione dati **duplicati**

Anomalia di **eliminazione**:

- eliminazione di righe → **perdita** di dati necessari per altre righe future

Anomalia di **modifica**:

- modifica dei dati in una riga → le modifiche ad **altre righe** a causa della duplicazione

Regola generale: una tabella non deve riguardare più di un tipo di entità.

Esempio Anomalie



EMPLOYEE2

<u>EmplID</u>	Name	DeptName	Salary	CourseTitle	DateCompleted
100	Margaret Simpson	Marketing	48,000	SPSS	6/19/2015
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/2015
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/2015
110	Chris Lucero	Info Systems	43,000	Visual Basic	1/12/2015
110	Chris Lucero	Info Systems	43,000	C++	4/22/2015
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/2015
150	Susan Martin	Marketing	42,000	Java	8/12/2015
100	Margaret Simpson	Marketing	48,000	JAVA	6/19/2016

Eliminazione

Se eliminassi il dipendente 140 non saprei più che nella mia azienda si tiene un corso di Tax Accounting

Modifica

Se cambiassi il salario del dipendente 110 dovrei aggiornarlo ovunque

Inserimento

Se volessi inserire un nuovo corso, oppure un nuovo dipendente, dovrei mettere a Null tutti i campi per i quali non sono disponibili informazioni. Inoltre, creerei inevitabilmente dei duplicati

Anomalie in questa tabella



- **Inserimento:** non è possibile immettere un nuovo dipendente senza che il dipendente assuma un corso (o almeno campi vuoti di informazioni sul corso)
- **Eliminazione:** se rimuoviamo il dipendente 140, perdiamo informazioni sull'esistenza di corso di Tax Acc
- **La modifica:** l'aumento di stipendio al dipendente 100 ci costringe ad aggiornare più (2) record

Perché esistono queste anomalie?

Poiché in questa relazione sono presenti due temi (tipi di entità). Ciò comporta la duplicazione dei dati e una dipendenza non necessaria tra le entità.

Risoluzione delle anomalie e 1FN



- Il problema della relazione EMPLOYEE2 è che contiene dati su due entità: DIPENDENTE e CORSO.
- Useremo la teoria della normalizzazione (descritta più avanti) per dividere EMPLOYEE2 in due relazioni.
- Una delle relazioni risultanti è EMPLOYEE1.
- L'altra la chiameremo EMP COURSE. La chiave primaria di questa relazione è la combinazione di EmpID e CourseTitle
- EMP COURSE è privo dei tipi di anomalie descritte in precedenza e quindi ben strutturato.

EMPLOYEE1			
<u>EmpID</u>	Name	DeptName	Salary
100	Margaret Simpson	Marketing	48,000
140	Allen Beeton	Accounting	52,000
110	Chris Lucero	Info Systems	43,000
190	Lorenzo Davis	Finance	55,000
150	Susan Martin	Marketing	42,000

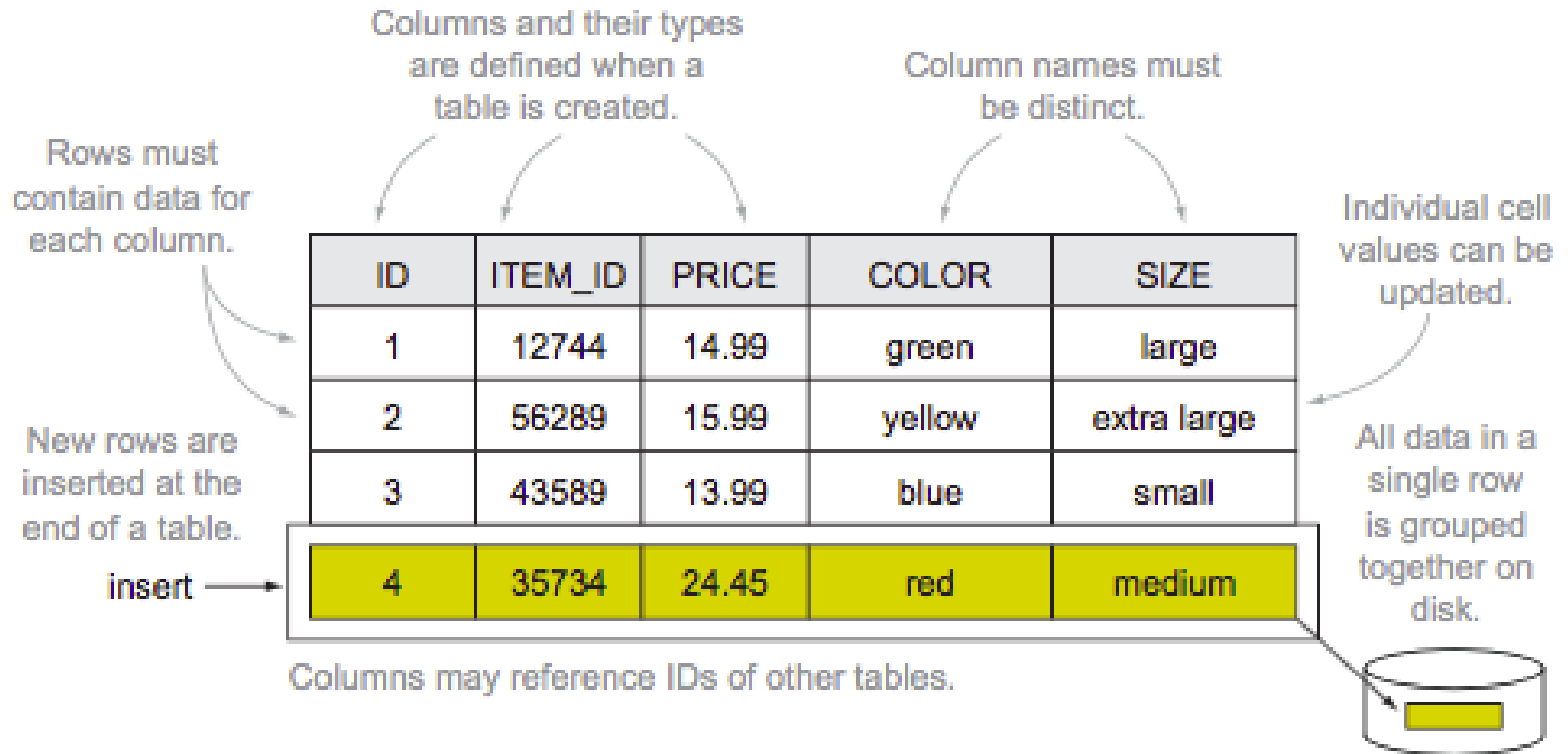
<u>EmpID</u>	<u>CourseTitle</u>	DateCompleted
100	SPSS	6/19/2015
100	Surveys	10/7/2015
140	Tax Acc	12/8/2015
110	Visual Basic	1/12/2015
110	C++	4/22/2015
150	SPSS	6/19/2015
150	Java	8/12/2015



3rd Lesson - NoSQL

- From RDBMS to NoSQL

RDBMS – row store



RDBMS - views



We want to restrict
general access to this
column.

Physical table

ORDER_ID	ORDER_DATE	SHIP_STATUS	CARD_INFO	TOTAL
123	2012-07-11	SHIPPED	VISA-1234...	39.45
124	2012-07-12	BACKORDER	MC-5678...	29.37
125	2012-07-13	SHIPPED	AMEX-9012...	42.47

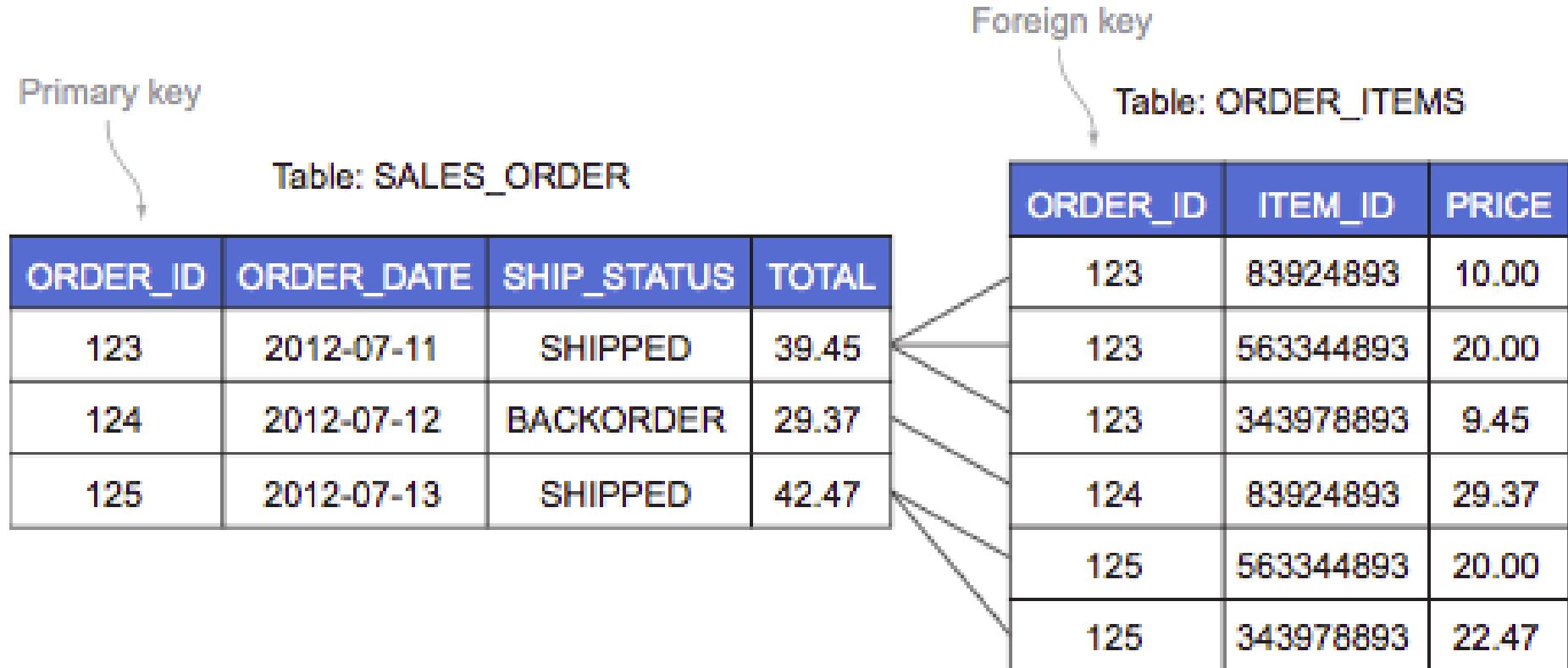
The physical table includes all the column, including credit card info. Only select users ever see the physical table.

View of table

ORDER_ID	ORDER_DATE	SHIP_STATUS	TOTAL
123	2012-07-11	SHIPPED	39.45
124	2012-07-12	BACKORDER	29.37
125	2012-07-13	SHIPPED	42.47

The view excludes some fields like credit card information. All sales analysts have access to the views.

RDBMS - JOINS



- Issues with scaling up when the dataset is just too big
- RDBMS were not designed to be distributed
- Began to look at multi-node database solutions
- Different approaches include:
 - Master-slave
 - Sharding

Scaling RDBMS – Master/Slave vs Sharding



- Master-Slave
 - All writes are written to the master. All reads performed against the replicated slave databases
 - Critical reads may be incorrect as writes may not have been propagated down
 - Large data sets can pose problems as master needs to duplicate data to slaves
- Partition or sharding
 - Scales well for both reads and writes
 - Not transparent, application needs to be partition-aware
 - Can no longer have relationships/joins across partitions
 - Loss of referential integrity across shards

RDBMS Limits



Feature	Strength	Weakness
Joins between tables	New views of data from different tables can easily be created.	All tables must be on the same server to make joins run efficiently. This makes it difficult to scale to more than one processor.
Transactions	Defining begin point, end point, and completion of critical transactions in an application is simple.	Read and write transactions may be slowed during critical times in a transaction unless the transaction isolation level is changed.
Fixed data definitions and typed columns	Easy way to define structure and enforce business rules when tables are created. You can verify on insert that all data conforms to specific rules. Allows range indexes over columns.	Difficult to work with highly variable and exception data when adding to a column.
Fine-grained security	Data access control by row and column can be done with a series of view and grant statements.	Setup and testing security access for many roles can be a complex process.
Document integration	None. Few RDBMSs are designed to easily query document structures.	Difficult to create reports using both structured and unstructured data.

- Not Only SQL or NO! SQL
- **A more general definition... a datastore that does not follow the relational model including using SQL to interact with the data.**
- Why?
 - One size does not fit all
 - Relational Model has scaling issues
 - Freedom from the tyranny of the DBA?

How did we get here?



- Explosion of social media sites (Facebook, Twitter) with large data needs
- Rise of cloud-based solutions such as Amazon S3 (simple storage solution)
- A shift to dynamically-typed data with frequent schema changes
- Open-source community

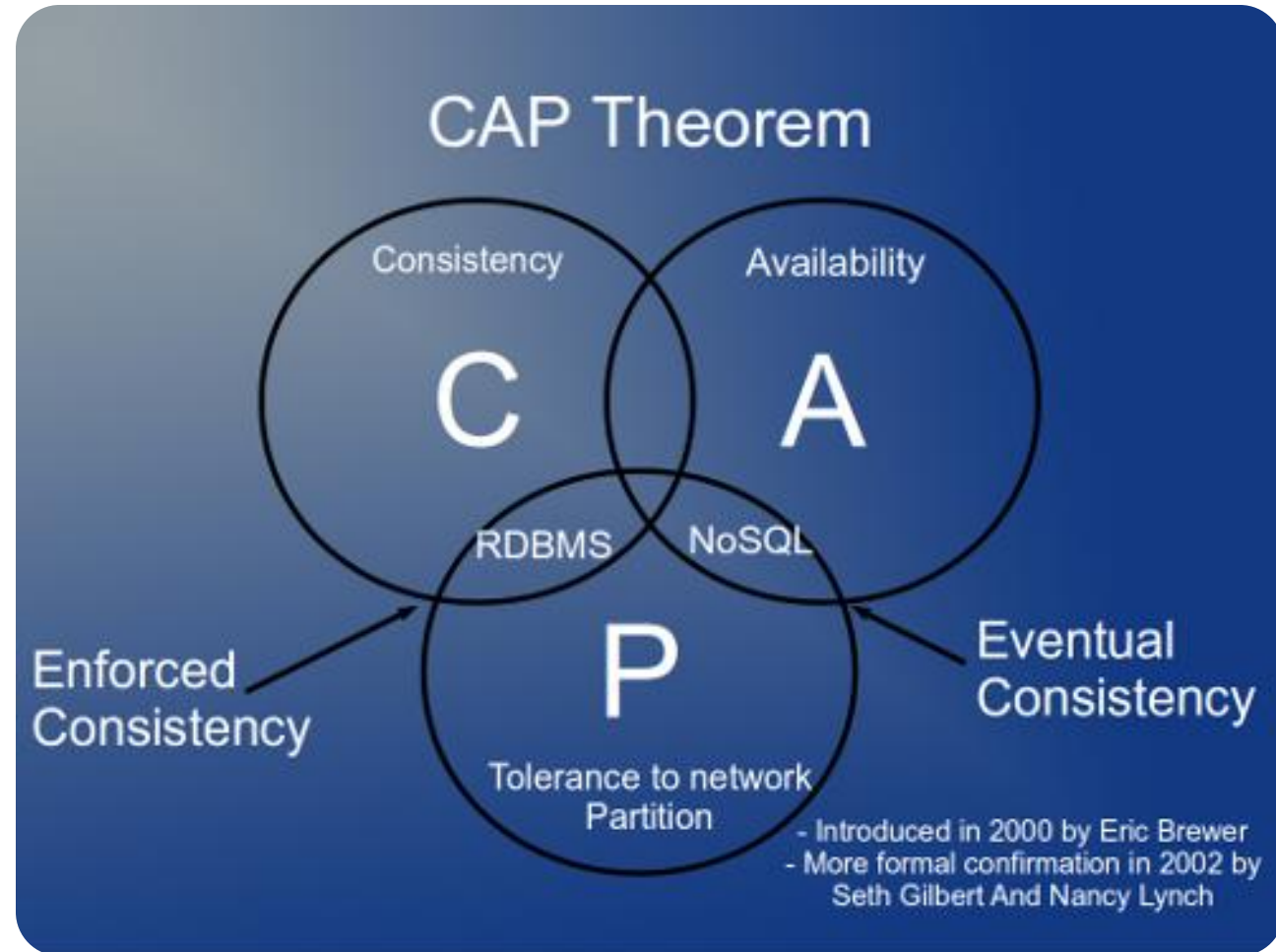
- **Schema-less** data storage
- Non-relational
- Flexible schema
- Other or additional query languages than SQL
- Easily distributed – **horizontal scaling**
- Unstructured data
- Supports **big data**
- Quick data storage and traversal
- Better performance

- Three major papers were the seeds of the NoSQL movement
 - [BigTable \(Google\)](#)
 - [Dynamo \(Amazon\)](#)
 - ❖ Gossip protocol (discovery and error detection)
 - ❖ Distributed key-value data store
 - ❖ Eventual consistency
 - CAP Theorem *Eric Brewer 2000*

CAP theorem



It is impossible for a [distributed data store](#) to simultaneously provide more than two out of the following three guarantees



CAP theorem



- **Consistency:** write a value and then you read the value you get the same value back.
- **Availability:** may not always be able to write or read. The system will say you can't write because it wants to keep the system consistent.
- **Partitionability:**
 - Any guarantee of consistency or availability is still guaranteed even if there is a partition.
 - If a system is not partition-tolerant, that means that if the network can lose messages or any nodes can fail, then any guarantee of atomicity or consistency is voided.

To scale you have to partition, so you are left with choosing either high consistency or high availability for a particular system. You must find the right

Consistency Model



- A consistency model determines rules for visibility and apparent order of updates.
- For example:
 - Row X is replicated on nodes M and N
 - Client A writes row X to node N
 - Some period of time t elapses.
 - Client B reads row X from node M
 - Does client B see the write from client A?
 - **For NoSQL, the answer would be: maybe (EVENTUAL CONSISTENCY)**
 - CAP Theorem states: Strict Consistency (**ACID**) can't be achieved at the same time as availability and partition-tolerance.

- **Atomicity**

- Requires that each transaction be "all or nothing": if one part of the transaction fails, then the entire transaction fails;

- **Consistency**

- Ensures that any transaction will bring the database from one valid state to another.

- **Isolation (Concurrency Control)**

- Ensures that the concurrent execution of transactions results in a system state that would be obtained if transactions were executed sequentially, i.e., one after the other.

- **Durability**

- Ensures that once a transaction has been committed, it will remain so, even in the event of power loss, crashes, or errors.

Eventual Consistency



- When no updates occur for a long period of time, eventually all updates will propagate through the system and all the nodes will be consistent
- For a given accepted update and a given node, eventually either the update reaches the node or the node is removed from service
- Known as BASE (**B**asically **A**vailable, **S**oft state, **E**ventual consistency), as opposed to ACID

- BASE stands for these concepts:
- **Basic Availability** allows systems to be temporarily inconsistent so that transactions are manageable. In BASE systems, the information and service capability are “basically available.”
- **Soft-state** recognizes that some inaccuracy is **temporarily** allowed and data may change while being used to reduce the amount of consumed resources.
- **Eventual consistency** means eventually, when all service logic is executed, the system is left in a consistent state.

- The number-one objective is to **allow new data to be stored**, even at the risk of being out of sync for a short period of time. They relax the rules and allow reports to run even if not all portions of the database are synchronized.
- They're *optimistic* in that they assume that eventually all systems will catch up and become consistent.
- BASE systems tend to be **simpler** and **faster** because they don't have to write code that deals with locking and unlocking resources.
- Their mission is to **keep the process moving** and deal with broken parts at a later time.
- BASE systems are ideal for **web storefronts**, where filling a shopping cart and placing an order is the main priority.

Pros and Cons of RDBMS



- RDBMS pros:
 - ACID transactions
 - Fine-grained security on columns and rows using views
 - Most SQL code is portable to other SQL databases, including open source options.
 - Typed columns and constraints will validate data before it's added to the database and increase data quality.
 - Existing staff members are already familiar with entity-relational design and SQL.
- RDBMS cons:
 - The object-relational mapping layer can be complex.
 - Entity-relationship modeling must be completed before testing begins, which slows development.
 - RDBMSs don't scale out when joins are required.
 - **Sharding over many servers can be done but requires application code and will be operationally inefficient.**
 - **Full-text search requires third-party tools.**

Pros and Cons of NoSQL



- NoSQL pros:
 - Loading test data can be done with drag-and-drop tools **before ER modeling is complete.**
 - Modular architecture allows components to be exchanged.
 - **Linear scaling** takes place as new processing nodes are added to the cluster.
 - Lower operational costs are obtained by autosharding.
 - Integrated search functions provide high-quality ranked search results.
 - There's no need for an object-relational mapping layer.
 - It's easy to store high-variability data.
- NoSQL cons:
 - **ACID transactions can be done only within a document at the database level. Other transactions must be done at the application level.**
 - **Document stores don't provide fine-grained security at the element level.**
 - NoSQL systems are new to many staff members and additional training may be required.
 - The **document store has its own proprietary nonstandard query language**, which prohibits portability.
 - The document store won't work with existing reporting and OLAP tools.